

融合特征匹配与异常行为分析的网络攻击检测系统

信息安全科技创新 · 课程大作业报告

韩宇飞 李哲 曾子恒 陈志恒 姜新晨

上海交通大学

2026 年 7 月 29 日

摘要

随着 Web 应用、远程运维与内网办公系统的普及，SQL 注入、跨站脚本（XSS）、暴力破解、端口扫描与异常外联等攻击仍是现实网络中常见的威胁类型。单一检测范式无论仅依赖静态特征库，还是仅依赖流量统计阈值，都难以兼顾已知攻击覆盖与未知行为发现。本课程大作业设计并实现一套**融合特征匹配与异常行为分析的网络攻击检测系统**，将**误用检测（Misuse Detection）**与**异常行为检测（Anomaly Detection）**融合，并提供统一告警汇总与图形化展示能力。

本文系统介绍我们五人小组完成的工程实现。整体架构为：**数据包捕获** → **协议解析/TCP 重组** → **三路并行检测** → **告警汇总与协同关联** → **双形态 GUI / 实时演示**。地基模块 capture（李哲）基于 scapy 将原始报文标准化为 **Packet Record JSON**；检测层由 **signature_engine**（曾子恒）、**bruteforce_detect**（陈志恒）、**anomaly_detect**（姜新晨）组成；展示与统筹由 **gui_alert**（韩宇飞）负责，含 **aggregator.py**、**correlator.py**、**gui.py**、**web_gui.py** 及 **demo/** 真实攻击演示套件。项目在 Phase1 即交付 **mock_data/mock_packets.json** 与 **docs/interface_spec.md**，PR #1 于 2026-07-13 首个合并。

截至 2026-07 联调阶段（main @ be0c01e），mock 已从 108 条扩充至 426 条，特征规则 28 条（SIG-001-028）；全量测试为 157 项通过、2 项因未安装 scapy 跳过。全链路 **python main.py --input mock_data/mock_packets.json** 实测产出 30 条汇总告警（signature 16 + bruteforce 3 + anomaly 11），correlator 完成 6 条跨检测器关联与 4 条严重度升级。本工作验证了「Schema 先行 + 模块化接口 + Mock 驱动联调」在多人协作场景中的有效性。

关键词：网络入侵检测；误用检测；异常行为检测；scapy 协议解析；TCP 流重组；特征匹配；暴力破解；端口扫描；告警汇总；多人协作工程

目录

1	引言	3
1.1	研究背景与问题陈述	3
1.2	项目目标与验收标准	3
1.3	小组分工与协作机制	3
1.4	报告结构	3
2	威胁模型与检测问题定义	4
2.1	系统边界、假设与不在范围	4
2.2	三层检测面	4
2.3	数据对象形式化	5
2.4	Mock 场景设计原则	5
3	系统总体架构与接口规范	6
3.1	逻辑分层与数据流	6
3.2	仓库目录结构 (main @ be0c01e)	6
3.3	Packet Record 完整字段	6
3.4	Alert Record 完整字段	7
3.5	汇总层扩展字段 (merged_alerts.json)	7
3.6	异常与边界约定	7
4	系统实现	9
4.1	Phase 1: 地基模块 capture (李哲)	9
4.2	Phase 2: 特征匹配引擎 signature (曾子恒)	10
4.3	Phase 2: 暴力破解检测 bruteforce (陈志恒)	11
4.4	Phase 2: 异常行为检测 anomaly (姜新晨)	13
4.5	Phase 3: 测试、日志与健壮性	14
4.6	Phase 4-5: 汇总、GUI、演示与 mock 扩充 (韩宇飞)	14
5	实验与检测效果评估	14
5.1	实验环境与数据集	14
5.2	实验设计方法	15
5.3	实验一: Phase1 基线 mock (108 条)	15
5.4	实验二: main 分支全链路 (426 条, 实测)	16
5.5	实验三: 汇总层与 correlator 协同效果	17
5.6	实验四: 自动化测试覆盖	17
5.7	实验五: 误报与边界分析	18
5.8	复现步骤	18
6	告警汇总、GUI 与跨模块协同	18
6.1	汇总管线架构	18
6.2	correlator 攻击阶段映射	18
6.3	GUI 与 Web 监控面板	19
6.4	汇总告警示例 (merged_alerts.json 摘录)	19
7	讨论	20
7.1	工程经验与协作成效	20
7.2	系统局限	20
7.3	改进方向	20
8	相关工作	21
8.1	规则型 IDS: Snort 与 Suricata	21
8.2	连接级分析: Zeek (原 Bro)	21
8.3	误用检测与异常检测的理论框架	21
8.4	与本项目的定位对比	21
9	结论	22
A	开发里程碑	23
B	五成员交付物清单	23
C	Verification	24

1 引言

1.1 研究背景与问题陈述

网络攻击检测 (Network Intrusion Detection, NID) 是保障信息系统可用性与数据完整性的基础能力。参考 OWASP Top 10 与 MITRE ATT&CK[4, 3], 本项目关注的攻击手法可粗分为两类:

1. **载荷型攻击**: 攻击者在 HTTP/TCP 等应用层载荷中嵌入恶意语义, 如 SQL 注入语句 UNION SELECT、XSS 脚本 `<script>alert(1)</script>`、Webshell 上传、远程命令执行等。此类攻击通常具有**可描述的字节级指纹**, 适合误用检测。
2. **行为型攻击**: 攻击者不依赖固定 payload, 而通过**频率、拓扑与时空模式**达成目的, 如针对 SSH 的暴力破解、针对内网网段的端口扫描、内网主机对陌生公网 IP 的异常外联、以及横向移动。此类攻击适合基线/statistics 检测。

商用 IDS/IPS (如 Snort、Suricata、Zeek) 通常集成 DPI、规则库与行为分析, 但课程项目受限于周期与人力, 需要在五人分工下交付一条**可演示、可测试、可扩展**的完整链路。核心工程挑战并非单一算法, 而是:

- 如何在抓包环境 (Linux root、WSL、网卡权限) 未就绪时, 不阻塞检测算法开发?
- 如何避免五人各自解析 pcap 导致字段语义不一致?
- 如何将三路异构告警汇总为 GUI 可消费的唯一格式?

1.2 项目目标与验收标准

表 1: 项目目标与对应交付物

编号	目标	验收交付物
G1	数据包捕获与协议解析	src/capture/、pcap/实时抓包 CLI
G2	特征串匹配引擎	config/signatures.txt、matcher.py
G3	暴力破解检测	login_monitor.py、时间窗口计数
G4	异常行为检测	baseline.py、四项检测、baseline_config.json
G5	告警汇总、协同与展示	aggregator.py、correlator.py、gui.py、web_gui.py、main.py
G6	Mock、测试与演示	mock_packets.json (426 条)、tests/、demo/

1.3 小组分工与协作机制

协作规范: (1) 每人仅修改 src/< 模块 >/ 与对应 tests/test_*.py; (2) 接口变更必须同步更新 docs/interface_spec.md 并在群内通知; (3) Phase 结束提交 PR, 由组长 Code Review 后合并 main。

1.4 报告结构

第 2 节定义威胁模型与数据对象; 第 3 节描述总体架构与接口规范; 第 4 节按 Phase 详述五模块实现; 第 5 节给出实验与检测效果; 第 6 节讨论汇总、GUI 与跨模块协同; 第 7-9 节为

表 2: 成员分工、Git 分支与核心职责

成员	学号	分支	职责
韩宇飞 (组长)	524031910172	feature/gui-alert	接口规范、aggregator、correlator、GUI/Web、main.py、demo/
李哲	524031910017	feature/packet-capture	抓包、解析、重组、mock (地基)
曾子恒	523010910022	feature/signature-engine	特征库、匹配算法
陈志恒	524031910111	feature/bruteforce-detect	暴力破解检测
姜新晨	524031910769	feature/anomaly-detect	异常行为检测

讨论、相关工作与结论。

2 威胁模型与检测问题定义

2.1 系统边界、假设与不在范围

假设

- 输入为离线 pcap 或 mock JSON，系统为**检测型 (IDS)** 而非 inline 阻断型 (IPS)；
- 报文 timestamp 单调可信，可用于 60s/300s 滑动窗口；
- 实验流量在授权环境内产生，不对公网发起攻击性请求；
- 攻击特征可通过 payload 子串/正则或五元组统计观测。

不在范围

TLS 解密、IP 分片完整重组（加分项未完全实现）、零日无特征攻击（除可选 ML 加分）、复杂 NAT/负载均衡拓扑还原。

2.2 三层检测面

载荷语义层 (Signature)

误用检测：规则库 $R = \{r_1, \dots, r_n\}$ ，每条规则含模式 pat 与匹配方式 (literal/regex)。报文 p 命中当且仅当 $pat \subseteq payload(p)$ (或 regex 匹配)。

连接行为层 (Bruteforce)

对登录端口集合 P_{login} ，统计窗口 T 内三元组 $(src, dst, dport)$ 的连接尝试次数 c 。若 $c \geq \theta_{bf}$ 则告警。

拓扑统计层 (Anomaly)

(1) 端口扫描： $|unique(dport)| > \theta_{scan}$ ；(2) 异常外联： $src \in Internal \wedge dst \notin Internal$ ；(3) 横向扩散： $|unique(dst_ip)| > \theta_{lat}$ ；(4) **高频连接**：窗口内连接次数 $> \theta_{rate}$ (main 分支新增第四项检测)。

2.3 数据对象形式化

Packet Record

$$p = \langle t, f, s_{ip}, s_p, d_{ip}, d_p, proto, dir, flags, data, |data| \rangle$$

Alert Record

$$a = \langle id, det, cat, s_{ip}, s_p, d_{ip}, d_{net}, d_p, sev, desc, evid, t \rangle$$

其中 $det \in \{signature, bruteforce, anomaly\}$, $sev \in \{low, medium, high\}$ 。

统一检测契约

```
def detect(packets: list[dict], ...) -> list[dict]:
    # 输入: Packet Record 列表; 其余可选参数由各模块定义
    # 输出: Alert Record 列表; 无告警时必须返回 [], 不得返回 None
```

接口规范以 config 作为统一可选参数示例; 实际代码中 signature 接收可选规则列表, bruteforce 接收窗口、阈值和端口集合, anomaly 接收配置字典。三者均保持首参数与返回类型一致, 且额外参数均有默认值。

2.4 Mock 场景设计原则

Mock 数据是 Phase1-3 的可执行接口契约。每条记录通过源 IP 编码场景语义, 便于 B/C/D 自测:

表 3: Mock 场景—源 IP 编码—预期检测模块

场景	代表 src_ip	关键可观测特征	detector
正常 HTTP 等	192.168.1.10	常规 GET/SSH/FTP	无
SQL 注入	192.168.1.10	UNION SELECT 等	signature
XSS	192.168.1.11	<script>	signature
木马/命令执行	192.168.1.12	faxsurvey 等	signature
SSH 暴力破解	192.168.1.99	18× SYN→22	bruteforce
端口扫描	192.168.1.77	24+ 不同 dport	anomaly
异常外联	192.168.1.55	内网 →203.0.113.x	anomaly
高频连接	192.168.1.88	120 次/60s	anomaly
慢速扫描	172.16.0.77	16 ports/60s	anomaly
DNS 隧道	10.0.0.5	SIG-023/024	signature
Webshell、XXE	多源 IP	SIG-013-022	signature

426 条扩充 mock 在 Phase1 七场景基础上新增: 并行攻击源、横向扩散临界 (172.16.0.99)、FTP 暴破临界 (192.168.1.98)、419 条 TCP 注入 tcp_seq/tcp_ack 以支持重组 v2 测试。

3 系统总体架构与接口规范

3.1 逻辑分层与数据流

系统采用五层架构，与 upstream main 仓库目录一一对应：

- 1. 采集层 `src/capture/`: scapy 抓包 → `parse_packet` → 可选 `reassemble` (v1/v2);
- 2. 数据层: Packet Record JSON (`mock_data/mock_packets.json` 或 `demo/live_capture.py` 输出);
- 3. 检测层: `signature_engine/`、`bruteforce_detect/`、`anomaly_detect/` 并行 `detect()`;
- 4. 汇总协同层 `gui_alert/`: `aggregator` 合并去重 + `correlator` 跨检测器关联/攻击链/严重程度升级;
- 5. 展示与演示层: `gui.py` (tkinter)、`web_gui.py` (Python 标准库 `http.server`, 端口 8099)、`demo/` (靶机 + 攻击脚本)。

Phase1-3 使用 mock; Phase4 起输入切换为 `results/live_capture.json` 或 pcap 解析输出, **Packet Record 格式不变**。全链路入口为 `main.py`, 支持 `--input`、`--web`、`--watch N`、`--gui-only` 四种模式。

3.2 仓库目录结构 (main @ be0c01e)

表 4: 核心目录与模块映射

路径	职责
<code>main.py</code>	全链路入口 (韩宇飞)
<code>config/signatures.txt</code>	主分支 28 条规则; 规则库与匹配模块由曾子恒负责
<code>config/baseline_config.json</code>	四项异常阈值 (姜新晨)
<code>src/capture/</code>	抓包、解析、重组、mock 生成 (李哲)
<code>src/signature_engine/</code>	<code>signature_db.py</code> + <code>matcher.py</code>
<code>src/bruteforce_detect/</code>	<code>login_monitor.py</code>
<code>src/anomaly_detect/</code>	<code>baseline.py</code> + <code>anomaly_detector.py</code>
<code>src/gui_alert/</code>	<code>aggregator</code> 、 <code>correlator</code> 、 <code>gui</code> 、 <code>web_gui</code>
<code>demo/</code>	靶机、抓包、9 类攻击脚本、一键演示
<code>mock_data/mock_packets.json</code>	426 条场景化报文
<code>results/</code>	各模块告警 JSON + <code>merged_alerts.json</code>
<code>tests/</code>	6 个测试文件, 共收集 159 项 (157 通过、2 跳过)
<code>docs/interface_spec.md</code>	权威接口规范

3.3 Packet Record 完整字段

表 5: Packet Record 字段定义 (摘自 `interface_spec.md`)

字段	类型	必填	说明
<code>timestamp</code>	<code>string</code>	是	ISO8601 毫秒
<code>flow_id</code>	<code>string</code>	否	<code>src:port->dst:port/proto</code>
<code>src_ip /</code> <code>dst_ip</code>	<code>string</code>	是	IP 地址

字段	类型	必填	说明
src_port / dst_port	int/null	否	端口；ICMP 为 null
protocol	string	是	TCP/UDP/ICMP/ARP
direction	string/null	否	request/response
flags	string	否	TCP 标志 S/SA/PA 等
payload	string	否	应用层；二进制用 \xNN
payload_len	int	是	线路上原始字节长度
tcp_seq / tcp_ack	int/null	否	TCP 序列/确认号；426 mock 中 419 条 TCP 已注入

3.4 Alert Record 完整字段

表 6: Alert Record 字段定义

字段	类型	必填	说明
alert_id	uuid	是	全局唯一
behavior_id	uuid	否	行为事件 ID；aggregator 可自动关联
detector	string	是	signature/bruteforce/anomaly
category	string	是	如「Web 攻击/SQL 注入」
src_ip / dst_ip	string	是	源/受害 IP
src_port / dst_port	int/null	否	攻击源端口与受害目标端口
dst_network	string	否	CIDR，端口扫描等网段级告警
severity	string	是	low/medium/high
description	string	是	GUI 展示用
evidence	string	否	匹配串或统计证据
timestamp	string	是	ISO8601

3.5 汇总层扩展字段 (merged_alerts.json)

aggregator.aggregate() 在合并后调用 correlator，为每条告警**可选**追加：

3.6 异常与边界约定

输入文件缺失/为空 → warning + 空列表；非必填字段缺失 → 按 null 跳过；时间窗口数据不足 → 不告警不报错；flow_id 缺失时消费方自行构建五元组。

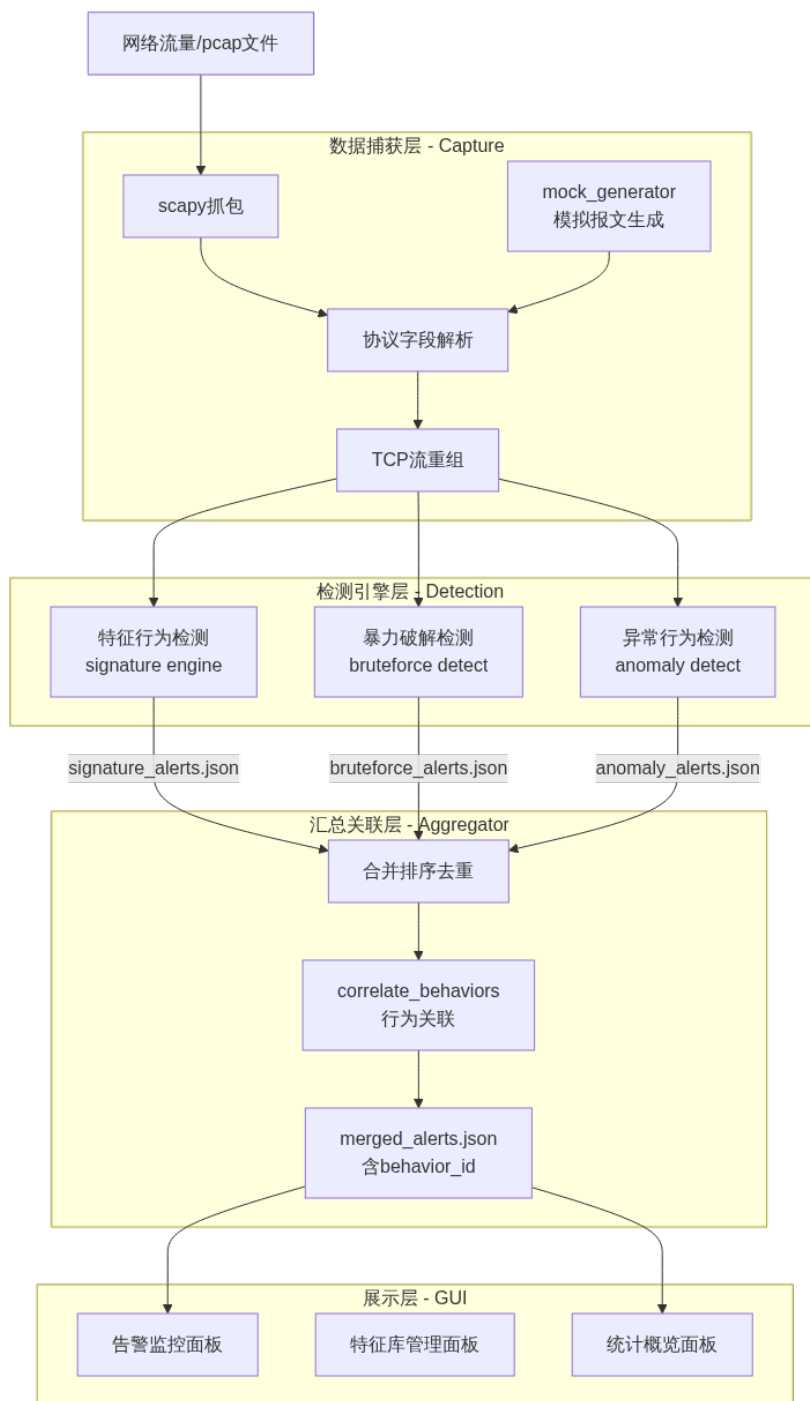


图 1: 系统分层架构与三路检测数据流

字段	说明
correlated_alerts	同源 IP 其他 detector 告警的 alert_id 列表
cross_detector_count	涉及的不同 detector 数量
attack_stage	六阶段: reconnaissance, exploitation, installation, c2, lateral_movement, credential_access
attack_chain_id	同一攻击链共享 UUID
original_severity / escalated	严重度升级前后对比

4 系统实现

4.1 Phase 1: 地基模块 capture (李哲)

合并: PR #1 (2026-07-13)。战略意义: 全组第一个合并 PR, 解除 B/C/D 对数据源的阻塞。

4.1.1 模块文件与职责

文件	职责
protocol_parser.py	scapy 报文 → Packet Record
tcp_reassembly.py	同流 payload 合并 (v1)
packet_capture.py	sniff/rdpcap、CLI、落盘 JSON
mock_generator.py	场景化 mock 合成
tests/test_capture.py	24 项 (含 TCP v2 重组)

protocol_parser.py 核心 API

函数	行为
encode_payload	二进制转义, payload_len 为原始字节数
infer_direction	SYN→request, SYN+ACK→response
build_flow_id	src:port→dst:port/PROTO 规范化
is_tls_traffic	检测 0x16 0x03 0x01, 标记 HTTPS 样本
parse_packet	scapy 包 → 完整 Packet Record dict

4.1.2 协议解析细节

parse_packet() 支持 ARP、IPv4+TCP/UDP/ICMP。核心函数:

- encode_payload(bytes): 可打印 ASCII 保留, 其余 \xNN; 返回 (str, payload_len);
- infer_direction(): SYN→request, SYN+ACK→response, 或知名端口启发式;
- build_flow_id(): 统一五元组字符串格式;
- is_tls_traffic() (加分项): 检测 TLS Handshake 0x16 0x03 0x01, 不解密。

4.1.3 TCP 流重组 v1 与 v2 演进

v1 (李哲交付): 按 flow_id 分组, 时间序拼接 payload; 控制包 (SYN/FIN/RST 无数据) 原样保留。

v2 (main 后续增强): 基于 tcp_seq/tcp_ack 双向追踪; 乱序 buffer、重传去重、seq 间隙 [MISSING N bytes] 标记; 无 seq 字段时降级为 v1 时间拼接。

4.1.4 抓包 CLI

抓包管线: sniff()/rdpcap → parse_packet × N → reassemble()。异常时返回空列表, 不抛未捕获异常。

```
python -m src.capture.packet_capture --pcap sample.pcap \
    --output results/captured_packets.json
python -m src.capture.packet_capture --live --interface eth0 --count 100
python -m src.capture.packet_capture --pcap x.pcap --no-reassemble
```

Packet Record 输出示例

```
{
  "timestamp": "2026-07-08T10:00:00.123",
  "flow_id": "192.168.1.10:51234->192.168.1.20:80/TCP",
  "src_ip": "192.168.1.10", "dst_ip": "192.168.1.20",
  "protocol": "TCP", "direction": "request", "flags": "PA",
  "payload": "GET /login.php?id=1 UNION SELECT ...",
  "payload_len": 68
}
```

4.1.5 Mock 数据初版 (108 条)

表 7: Phase1 mock_packets.json 场景统计

场景	条数	标识 IP
正常 HTTP/SSH/FTP/DNS/ICMP	22	192.168.1.10
SQL 注入	7	192.168.1.10
XSS	6	192.168.1.11
木马/恶意命令	4	192.168.1.12
SSH 暴力破解 (SYN+RST 对)	36	192.168.1.99
端口扫描	24	192.168.1.77
异常外联	3	192.168.1.55
正常 HTTPS (TLS 二进制)	1	192.168.1.10
合计	108	

4.2 Phase 2: 特征匹配引擎 signature (曾子恒)

合并: PR #3 (2026-07-22)。

特征库格式

config/signatures.txt 每行: 规则 ID | 攻击类型 | 匹配模式 | 特征串 | 适用协议 | 严重程度。支持 literal (大小写不敏感子串) 与 regex。

signature_db.py

加载、校验规则; add_signature/delete_signature 供 GUI 管理; regex 编译失败则跳过并 warning。

matcher.py 算法

逐报文遍历规则；协议过滤；命中后生成 Alert；**60 秒行为聚合**：同一 (src_ip, dst_ip, category) 多次命中合并为一条，避免 GUI 刷屏。

典型规则类别 (main 分支扩展后)

除 Phase1 的 SQL/XSS/木马外，main 分支扩展至 **28 条** (SIG-001-028)：Webshell (013-015)、路径遍历 (016-017)、命令注入 (018-020)、XXE (021-022)、DNS 隧道 (023-024)、SMB/C2 (025-028)。matcher.py 支持 URL 解码匹配 (curl %3C → <script>)，Web 规则协议字段须填 TCP 而非 HTTP。

告警输出示例 (SQL 注入)

```
{
  "detector": "signature",
  "category": "Web 攻击/SQL 注入",
  "severity": "high",
  "description": "检测到 SQL 注入特征: UNION SELECT",
  "evidence": "GET /login.php?id=1 UNION SELECT ..."
}
```

CLI

```
python -m src.signature_engine.matcher \
  --input mock_data/mock_packets.json \
  --output results/signature_alerts.json
```

4.3 Phase 2: 暴力破解检测 bruteforce (陈志恒)

提交与交付

陈志恒在 feature/bruteforce-detect 分支完成两次核心提交：ad8e877 实现 src/bruteforce_detect/login_monitor.py 的检测逻辑，c78abe2 完成 tests/test_bruteforce.py 的 25 项测试与异常处理。模块输出 results/bruteforce_alerts.json，并严格对齐统一 Alert Record。

监控端口

$P_{login} = \{21, 22, 23, 3389, 3306, 5432, 6379, 8080, 8443\}$ ，覆盖 FTP、SSH、Telnet、RDP、MySQL、PostgreSQL、Redis 与常见 Web 登录端口。

检测逻辑

模块先过滤非 TCP 报文，并按 (src_ip, dst_ip, dst_port) 归集连接尝试。优先将 flags 含 S 且不含 A 的报文视为新建连接；若某分组完全缺少 SYN，则按 flow_id 去重后退化计数。目标主机从登录服务端口返回的 RST 单独累计，作为连接遭拒或认证失败的辅助证据。

对排序后的事件时间戳使用双指针滑动窗口，求任意 $T = 60s$ 内的最大尝试数；当最大值达到阈值 $\theta = 10$ 时产生一条聚合告警。尝试数达到 2θ 时严重度为 high，否则为 medium。

Evidence 采用 attempt_count=18, time_window_sec=60, threshold=10, 若存在拒绝响应则追加 rejected_conn。

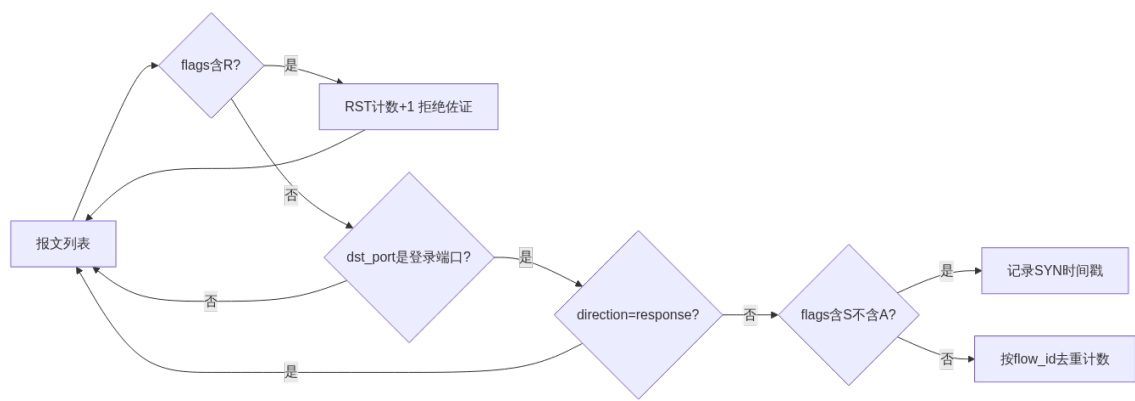


图 2: 暴力破解模块的连接尝试与 RST 证据归集流程

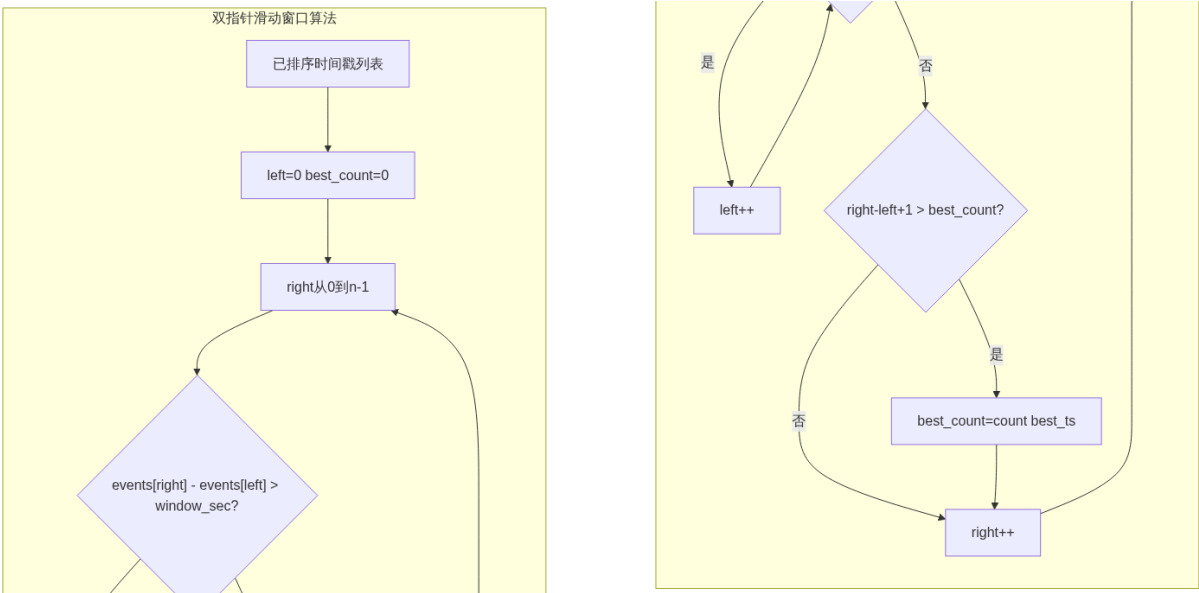


图 3: 双指针滑动窗口求最大连接尝试数

对 capture 的依赖

模块依赖 timestamp、源/目的 IP、目的端口与 TCP flags 的语义正确; direction=response 用于排除服务器回包, flow_id 用于缺少 SYN 时的退化计数。检测逻辑不读取 payload 或 payload_len。在 Phase1 mock 中, 192.168.1.99 的 18 条纯 SYN 及其时间分布构成基准检出场景。

告警输出示例（关键字段）

```
{
  "alert_id": "uuid-string",
  "behavior_id": "same-as-alert-id",
  "detector": "bruteforce",
  "category": " 暴力破解/非法登录",
  "src_ip": "192.168.1.99",
  "src_port": null,
  "dst_ip": "192.168.1.20",
  "dst_network": null,
  "dst_port": 22,
  "severity": "medium",
  "evidence": "attempt_count=18, time_window_sec=60, threshold=10, rejected_conn=18"
}
```

4.4 Phase 2: 异常行为检测 anomaly（姜新晨）

合并：PR #2（2026-07-22）。

模块组成

baseline.py: 加载 baseline_config.json、主机行为基线统计；anomaly_detector.py: 四项检测入口。

阈值配置 baseline_config.json

- 端口扫描：60s 内 $\text{unique}(\text{dst_port}) > 15$ （自 20 下调）；
- 异常外联：内网 CIDR 127.0.0.0/8, 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12；
- 横向扩散：300s 内 $\text{unique}(\text{内网 dst_ip}) > 8$ （自 10 下调）；
- 高频连接：60s 内连接次数 > 80 （自 100 下调）。

告警特点

端口扫描填写 `dst_network="192.168.1.0/24"`；异常外联 evidence 含 `internal_networks` 列表。

四项检测算法概要

1. **端口扫描**：滑动窗口内按 `src_ip` 统计 $\text{unique}(\text{dst_port})$ ；
2. **异常外联**：内网源 IP 访问非内网 CIDR 目的 IP；
3. **横向扩散**：窗口内访问 $\text{unique}(\text{内网 dst_ip})$ 超阈值；
4. **高频连接**：窗口内单 IP 连接总数超 `max_connections_per_ip`。

灵敏度调优（426 条 mock 阶段）

为覆盖「临界样本」答辩演示，main 分支将端口扫描阈值从 20 降至 15、横向扩散从 10 降至 8，并新增慢速扫描（172.16.0.77，16 端口）、中等频率连接（192.168.1.166，90 次/60s）等场景，使旧阈值下漏报的样本在新阈值下可稳定检出。

4.5 Phase 3: 测试、日志与健壮性

主分支共收集 159 项测试，其中 157 项通过，2 项 capture 测试因当前环境未安装 scapy 而跳过。通过项分布为 capture 24、signature 38、bruteforce 25、anomaly 45、GUI/aggregator 25；各模块统一 logging 格式，输入缺失或为空时返回 [] 而不抛出未处理异常（interface_spec §5）。

4.6 Phase 4-5: 汇总、GUI、演示与 mock 扩充（韩宇飞）

main.py 运行模式

```
python main.py --input mock_data/mock_packets.json # 单次全链路
python main.py --web # Web 面板 :8099
python main.py --watch 3 --input results/live_capture.json # 持续检测
python main.py --gui-only # tkinter 读已有告警
```

aggregator.py + correlator.py

aggregate(): 加载三路 JSON → 排序去重 → correlate_behaviors() (60s 窗口 behavior_id) → correlator 三函数 → merged_alerts.json。

双形态 GUI

gui.py: tkinter 桌面版，特征库增删改查；web_gui.py: Mock/Live 双模式、2s 自动刷新、清空 + 基线机制。

demo/ 真实攻击演示

target_server.py (有漏洞 HTTP 靶机)、live_capture.py (Windows Npcap 抓包)、attack_menu.sh (Kali 9 类攻击菜单)、start_demo.bat (Windows 一键启动)。支持跨机器 Kali→Windows 答辩演示。

Mock 演进 108 → 426

新增 Webshell/XXE/DNS 隧道/SMB、并行攻击源、慢速扫描 (172.16.0.77)、横向临界 (172.16.0.99)、高频连接 (.88 120 次/.166 90 次)；scripts/add_seq_to_mock.py 为 TCP 注入 tcp_seq/tcp_ack。

5 实验与检测效果评估

本节全部数据来自上游 main 分支本地复现实验。检索与运行日期为 2026-07-28，最新 commit 为 be0c01e (README 日程同步)。仓库地址为 https://github.com/pnnooy/network_detection，全链路命令见本节末的复现步骤。

5.1 实验环境与数据集

数据集概况(426 条 mock): TCP 419 / UDP 5 / ICMP 2; 419 条 TCP 含 tcp_seq/tcp_ack; direction 分布 request 377 / response 49。高频源 IP 包括 192.168.1.88 (120 包/60s, 高频连

表 8: 实验软硬件环境

项目	配置
Python	3.13 (要求 ≥ 3.9)
依赖	scapy、pytest; Web GUI 使用 Python 标准库 <code>http.server</code>
操作系统	Windows 10/11
输入数据	<code>mock_data/mock_packets.json</code> (426 条)
特征规则	<code>config/signatures.txt</code> (28 条, SIG-001-028)
基线阈值	<code>config/baseline_config.json</code> (端口扫描 15/60s 等)
上游 commit	<code>be0c01e @ 2026-07-27</code>

接临界)、192.168.1.166 (90 包)、192.168.1.77 (24 端口扫描)、192.168.1.99 (18 SYN 暴破) 等, 覆盖 README 定义的 20+ 攻击场景。

5.2 实验设计方法

我们采用三层验证策略:

1. **单元测试**: 各模块 `tests/test_*.py` 验证算法边界与 JSON Schema 合规;
2. **分模块端到端**: 对 426 条 mock 分别运行 `signature/bruteforce/anomaly` 的 `detect()`, 统计告警条数与类别分布;
3. **全链路集成**: `main.py` 驱动三路检测 $\rightarrow$ `aggregator.aggregate()` $\rightarrow$ `merged_alerts.json`, 验证 correlator 协同字段 (`correlated_alerts`、`attack_stage`、`escalated`)。

评价指标: 检出率 (预期场景是否产生对应 `detector/category` 告警)、误报率 (正常流量 192.168.1.10 常规 GET 是否零告警)、聚合效果 (逐包命中 $\rightarrow$ 行为告警压缩比)、跨检测器关联数。

5.3 实验一: Phase1 基线 mock (108 条)

Phase1 由李哲交付的 108 条 mock 构成「接口契约测试集」, PR #1 合并后成为全组开发基准。

表 9: Phase1 特征匹配结果 (PR #3 自测, 108 条 mock)

指标	结果
报文数	108
加载规则	12 (SIG-001-012)
特征逐包命中	17
60s 行为聚合后告警	4
正常流量误报	0
capture 单测	22 passed

聚合后 4 条行为告警分别对应: **SQL 注入** (7 次 UNION SELECT 等命中合并为 1 条)、**XSS** (6 次 `<script>` 合并)、**木马通信** (`faxsurvey`)、**恶意命令** (`/bin/cat%20` 等)。该实验证明了「逐包检测 + 时间窗口聚合」可将 GUI 展示从 17 条碎片告警压缩为 4 条可解释行为事件。

5.4 实验二：main 分支全链路（426 条，实测）

在 be0c01e 上运行 `python main.py --input mock_data/mock_packets.json`，得到如下**实测**结果（非估算）：

表 10: 全链路检测实测摘要（426 条 mock，2026-07-28）

指标	实测值
输入报文	426
特征规则加载	28 条
特征逐包命中	37 → 聚合后 16 条
暴力破解告警	3
异常行为告警	11
汇总 merged_alerts.json	30
跨检测器关联 (≥2 detector)	6 条
攻击链阶段标注	30/30
严重度升级	4 条
pytest 全量	157 passed, 2 skipped

5.4.1 特征引擎 (signature) 明细

28 条规则覆盖 SQL 注入 (5)、XSS (2)、木马 (1)、恶意命令 (4)、Webshell (3)、路径遍历 (2)、命令注入 (3)、XXE (2)、DNS 隧道 (2)、SMB/C2 (4)。扫描 426 包产生 37 次逐包命中，经 60s (src_ip, dst_ip, category) 窗口聚合为 **16 条**行为告警，压缩比约 2.3:1。

典型聚合告警示例 (SQL 注入): `description="60 秒内 192.168.1.10 对 192.168.1.20 命中 7 次 SQL 注入特征"`；evidence 含 `hit_count=7`, `patterns=[UNION SELECT, ' OR 1=1, ...]`。

5.4.2 暴力破解 (bruteforce) 明细

表 11: 暴力破解告警实测（窗口 60s，阈值 10）

src_ip	目标	尝试次数	说明
192.168.1.99	192.168.1.20:22	18 SYN	SSH 暴破 (Phase1 基准场景)
192.168.1.98	192.168.1.20:21	10 SYN	FTP 暴破 (临界阈值)
192.168.1.201	192.168.1.30:22	12 SYN	扩充 mock 新增源

算法依据 flags 含 S 的 SYN 包在 (src_ip, dst_ip, dst_port) 三元组上滑动计数；mock 中 192.168.1.99 的 18 条纯 SYN 及正确 timestamp 分布是检出的关键依赖。

5.4.3 异常检测 (anomaly) 明细

阈值取自 baseline_config.json：端口扫描 `unique_dst_port_threshold=15`（自 20 下调）、横向扩散 `internal_dst_count_threshold=8`（自 10 下调）、高频连接 `max_connections_per_ip=80`（自 100 下调），使慢速扫描（172.16.0.77，16 端口）等临界样本可稳定检出。

表 12: 异常行为告警分类实测 (11 条)

类别	条数	代表 src_ip	关键统计量
端口扫描	3	192.168.1.20 / .77 / 172.16.0.77	18/24/16 unique dport $\geq$ 15
异常外联	4	.10 / .55	内网 $\rightarrow$ 8.8.8.8 / 203.0.113.x
内网横向扩散	2	10.0.0.77 / 172.16.0.99	15/9 unique 内网 dst $\geq$ 8
异常高频连接	2	.88 / .166	120/90 次/60s $\geq$ 80

5.5 实验三：汇总层与 correlator 协同效果

aggregator.aggregate() 执行: 加载三路 JSON $\rightarrow$ 按 timestamp 排序 $\rightarrow$ alert_id 去重 $\rightarrow$ correlate_behaviors() (60s 窗口, 同源同类共享 behavior_id) $\rightarrow$ correlator 三函数。

表 13: correlator 协同输出统计

指标	值
汇总告警总数	30
severity 分布	high 16 / medium 14
跨检测器关联 (cross_detector_count $\geq$ 2)	6
攻击阶段标注覆盖	30/30
攻击链 ID 分配	20 条链
严重度升级 (escalated=true)	4

跨检测器关联案例: 192.168.1.10 同时触发 signature (SQL 注入, attack_stage=exploitation) 与 anomaly (异常外联 8.8.8.8:53, attack_stage=c2)。两条告警通过 correlated_alerts 互引, cross_detector_count=2, 外联告警由 medium 升级为 high (escalated=true)。192.168.1.55 同时有 anomaly 异常外联 (3 条) 与 signature 命令注入, 形成「C2 外联 + 载荷利用」组合证据。

攻击阶段分布: exploitation 14、reconnaissance 5、c2 4、credential_access 3、installation 2、lateral_movement 2, 覆盖 MITRE 风格六阶段映射 (见 correlator.STAGE_MAP)。

5.6 实验四：自动化测试覆盖

表 14: pytest 模块覆盖 (main @ be0c01e)

测试文件	覆盖模块	用例数
test_capture.py	TCP v1/v2 重组、协议解析	24
test_signature.py	规则加载、匹配、聚合	38
test_bruteforce.py	滑动窗口、SYN 计数	25
test_anomaly.py	四项异常检测	45
test_gui_aggregator.py	汇总、correlator、GUI	25
通过合计		157

本次 Windows 环境运行 pytest tests/ -q 得 157 passed, 2 skipped。两项跳过均来自

test_capture.py，原因为环境中未安装 scapy；测试文件实际共收集 159 项。

5.7 实验五：误报与边界分析

192.168.1.10 的 28 条混合流量包含正常 HTTP/SSH/FTP/DNS 与 7 条 SQL 注入载荷；signature 对注入样本告警，其余正常载荷不产生特征误报。与此同时，anomaly 会将访问 8.8.8.8:53 的常规 DNS 查询标记为「异常外联」，说明仅凭“内网到公网”判断会产生误报，生产环境需要目的地址白名单或域名信誉机制。

另一个边界现象来自 192.168.1.20：它向 SSH 爆破源返回的 18 条 RST 响应具有不同临时目的端口，当前端口扫描检测未过滤 direction=response，因而将服务器响应误判为一次 18 端口扫描。该结果仍计入当前 11 条 anomaly 告警，但属于需要在后续版本修复的交叉语义误报。192.168.1.98 的 FTP 数据共有 15 次 SYN，其中任意 60s 窗口的最大值为 10，恰好达到阈值，验证了边界条件采用“达到即告警”。

5.8 复现步骤

```
git clone https://github.com/pnnooy/network_detection.git
cd network_detection && pip install -r requirements.txt
python main.py --input mock_data/mock_packets.json
# 期望输出: merged_alerts.json 30 条告警
python -m pytest tests/ -q
python main.py --web # Web 面板 http://127.0.0.1:8099
```

6 告警汇总、GUI 与跨模块协同

6.1 汇总管线架构

三路检测器各自输出独立 JSON，汇总层不修改检测逻辑，仅做**时序合并 + 语义增强**：

1. aggregate(alert_files)：加载 signature/bruteforce/anomaly_alerts.json；
2. 按 timestamp 升序排列，按 alert_id UUID 去重；
3. correlate_behaviors()：同一 (src_ip, category) 在 60s 内告警共享 behavior_id，跨 detector 同源同类亦关联；
4. correlate_cross_detector()：同一 src_ip 被 ≥ 2 个 detector 检出时，填充 correlated_alerts 与 cross_detector_count；
5. detect_attack_chain()：按 STAGE_MAP 标注 attack_stage，多阶段同源 IP 共享 attack_chain_id；
6. escalate_severity()：多 detector 交叉验证时 low→medium→high 升一级；
7. save_merged() → results/merged_alerts.json。

网段级 anomaly 告警通过 dst_network="192.168.1.0/24" 保留扫描目标网段语义，避免压缩为单 IP 造成信息丢失。

6.2 correlator 攻击阶段映射

correlator.py 将 (detector, category) 映射至六阶段攻击链：

表 15: STAGE_MAP 节选 (main 分支)

阶段	detector	category 示例
reconnaissance	anomaly	端口扫描、异常高频连接
exploitation	signature	SQL 注入、XSS、命令注入、XXE、DNS 隧道
installation	signature	Webshell、木马通信
c2	anomaly	异常外联
lateral_movement	anomaly	内网横向扩散
credential_access	bruteforce	暴力破解/非法登录

答辩演示时可选取 192.168.1.55: 先展示 anomaly「异常外联」告警, 再展示 signature「命令注入」, 最后展示 correlator 在 evidence 中追加的 attack_chain: c2 → exploitation 字符串。

6.3 GUI 与 Web 监控面板

系统提供双形态展示:

tkinter 桌面版 (gui.py)

按 severity/category/detector 筛选; 双击查看 evidence; 特征库在线增删, 持久化 config/signatures.txt; python main.py --gui-only 直接读取已有 merged_alerts.json。

标准库 Web 版 (web_gui.py)

python main.py --web 启动, 浏览器访问 http://127.0.0.1:8099。支持 Mock/Live 双模式: Mock 模式重跑 426 条检测展示 30 条告警; Live 模式配合 demo/live_capture.py 每 2s 刷新, Kali 攻击时告警实时弹出。Web 面板含「清空 + 基线」机制, 避免历史告警重复计数。

真实攻击演示 (demo/)

README 定义跨机器演示: Windows 端 demo/start_demo.bat 启动靶机 + 抓包 + 检测 + Web; Kali 端 demo/attack_menu.sh 提供 9 类攻击脚本 (SQL 注入、XSS、端口扫描、SSH 爆破等)。该路径将 Phase4「真实 pcap 替换 mock」落到可答辩的 live 场景。

6.4 汇总告警示例 (merged_alerts.json 摘录)

```
{
  "detector": "anomaly", "category": "异常外联",
  "src_ip": "192.168.1.10", "dst_ip": "8.8.8.8", "dst_port": 53,
  "severity": "high", "original_severity": "medium", "escalated": true,
  "cross_detector_count": 2,
  "correlated_alerts": ["dda09e91-ad9a-4d80-811a-d8ec6e6d465b"],
  "attack_stage": "c2",
  "evidence": "dst_ip=8.8.8.8, ...; attack_chain: c2 -> exploitation"
}
```

7 讨论

7.1 工程经验与协作成效

Schema 先解除关键路径阻塞

李哲 Phase1 交付 Packet Record + 108 条 mock + `interface_spec.md`, PR #1 (2026-07-13) 为全组首个合并项。此后 24 小时内 B/C/D 均基于同一 JSON 启动 Phase2, 联调风险从 Phase4 前移至 Phase2.5「接口对齐检查」(7/20, 韩宇飞运行 aggregator 校验各模块 JSON 字段)。

Mock 即可执行契约

426 条 mock 每条通过源 IP 编码场景语义(如.99= 爆破、.77= 扫描、.55= 外联 +.10=SQL), 使测试失败可直接定位为「接口字段错误」或「算法阈值不当」, 而非「抓包环境不可复现」。108→426 的扩充由组长协调, 419 条 TCP 注入 `tcp_seq/tcp_ack` 以验证重组 v2。

行为聚合消除告警碎片化

signature 37 次逐包命中聚合为 16 条行为告警, GUI 从「逐包刷屏」变为「按攻击行为展示」, 契合项目「以行为为粒度」的设计目标(README § 一)。

教师建议的落地

main 分支 `correlator.py` 直接响应「多种检测思路融合」要求: 6 条跨检测器关联、20 条攻击链、4 条严重度升级, 可在答辩中演示「单模块漏报、多模块交叉验证补全证据链」。

踩坑与修复

README § 十四记录: `matcher.py` 增加 URL 解码(`curl %3C → <script>`)以匹配 XSS; `signatures.txt` 中 Web 规则须用 TCP 而非 HTTP (protocol 字段无 HTTP 枚举, 否则规则静默失效); Windows 抓包推荐 WSL/Npcap + `demo/live_capture.py`。

7.2 系统局限

表 16: 已知局限与影响范围

局限	说明
无 inline 阻断	检测型 IDS, 不产生 drop/reset
TLS 不解密	仅识别 Client Hello, 加密 payload 无法特征匹配
特征匹配复杂度	默认 $O(\text{报文数} \times \text{规则数})$, 28 规则可接受, 千级需 AC 自动机
Windows 抓包	scapy 原生兼容性受网卡驱动影响, 课程演示使用 Npcap/WSL 与 <code>demo/</code>
IP 分片	加分项未完整实现
DNS 8.8.8.8 外联	mock 故意构造, 生产环境需白名单

7.3 改进方向

1. 性能: AC 自动机多模式匹配, 对比 baseline 在 10k 规则下的吞吐;

2. **智能**: 可选 ML 基线 (Isolation Forest) 与传统阈值 A/B 对比;
3. **工程**: GitHub Actions 对 426 mock 做 CI 回归; CEF/JSON Lines 导出对接 SIEM;
4. **部署**: Web GUI 当前基于标准库 `http.server`, 可迁移至生产级 Web 框架并扩展 RBAC 与告警 ACK 工作流;
5. **抓包**: Phase4 live pcap 持续替换 mock, 验证 `tcp_reassembly v2` 在乱序/重传 pcap 上的增益。

8 相关工作

8.1 规则型 IDS: Snort 与 Suricata

Snort (Roesch, 1999) 与 Suricata 是工业界广泛部署的开源 IDS[2]。二者均采用「规则库 + 多模式匹配引擎 + 告警输出」架构, 与本项目 `signature_engine` 同属误用检测范式。差异在于: 工业系统支持 IP 重组、HTTP 规范化、预处理器链与 inline IPS 模式; 本项目简化为 Python + JSON 流水线, 但保留了「规则 ID + 严重度 + evidence」的可维护特征库结构。

8.2 连接级分析: Zeek (原 Bro)

Zeek 将网络流量转为结构化日志 (`conn.log`、`http.log`), 强调**连接而非逐包**作为分析单元。本项目的 Packet Record + `flow_id` + 60s 行为聚合借鉴此思想: 检测器消费的是「带流标识的结构化记录」, 而非原始 pcap 字节流。

8.3 误用检测与异常检测的理论框架

Denning (1987) 提出入侵检测模型, 区分基于已知攻击特征的误用检测与基于行为偏差的异常检测 [1]。本项目**同时实现**两类范式: signature (误用) + bruteforce/anomaly (行为统计), 并通过 correlator 在汇总层融合, 对应 Denning 框架中「多源证据综合」的工程化落地。

8.4 与本项目的定位对比

工业 IDS 与本项目的关系, 不宜简单理解为「缩小版 Snort」或「Python 版 Zeek」。二者在部署目标、工程约束与能力取舍上存在本质差异: 商用系统面向 7×24 生产流量与 inline 阻断, 课程项目则需在有限周期内交付**可测试、可演示、可分工协作**的完整链路。表 17 从五个维度概括差异, 并说明本项目的合理定位。

表 17: 工业 IDS 与课程项目的能力对比

能力	Snort/Suricata	Zeek	本项目
规则匹配	成熟	脚本化	28 条教学规模规则
行为统计	部分	强	爆破 + 扫描 + 外联 + 横向
攻击链关联	商业版	社区脚本	correlator 六阶段
GUI	第三方	第三方	tkinter + 标准库 Web
多人协作	不适用	不适用	Schema+Mock+Phase

在**规则匹配**维度, Snort/Suricata 支持千级规则、预处理器链与 IPS 模式; 本项目以 28 条 SIG 规则覆盖 SQL 注入至 DNS 隧道等教学规模场景, 重点验证「特征库 → 匹配 → 行为聚合

→ Alert Record」链路。在**行为统计**维度，Zeek 的连接级日志与脚本生态更为成熟；本项目的 bruteforce/anomaly 模块聚焦爆破、端口扫描、异常外联与内网横向四类可观测行为，阈值可通过 baseline_config.json 调优。

攻击链关联是本项目的差异化能力：商用产品通常依赖 SIEM 或商业关联引擎，本项目在汇总层以 correlator 实现 MITRE 风格六阶段标注、跨 detector 互引与严重度升级（实测 6 条交叉关联、20 条攻击链）。**GUI 与多人协作**两列则体现课程项目的工程特色，包括 tkinter + 标准库 Web 双形态展示，以及 Schema 先行、Mock 驱动、Phase 合并所支撑的五人并行开发模式。

综上，本项目定位于**贯通 IDS 核心数据流与双范式检测**的教学演示系统，而非追求与 Snort/ZeeK 在吞吐、规则规模或生产部署上的逐项对标；其价值在于证明团队理解 Packet Record 契约、三路检测互补及汇总层语义增强，并能在 main.py 单命令与 demo/ live 演示两条路径上复现端到端效果。

9 结论

本小组在 2026 年 7 月完成「融合特征匹配与异常行为分析的网络攻击检测系统」从数据包捕获、三路并行检测、告警汇总协同到双形态 GUI 的完整实现。项目在 main 分支（commit be0c01e）上通过 **426 条 mock 报文** → **30 条汇总告警**的全链路实测，以及 **157 项通过**、**2 项跳过的** pytest 自动化回归，验证了系统可用性。

主要贡献：

1. **可替换报文契约**：Packet Record JSON 解耦 capture 与检测，PR #1 成为全组关键路径上的第一个稳定数据源；
2. **三类互补检测**：28 条特征规则（误用）+ 3 条爆破告警（连接统计）+ 11 条异常告警（基线偏离），覆盖 SQL 注入至 DNS 隧道等 20+ 场景；
3. **跨检测器协同**：correlator 实现 6 条交叉关联、20 条攻击链、4 条严重度升级，提升告警可解释性；
4. **可复现演示路径**：python main.py --input mock_data/mock_packets.json 单命令端到端；demo/ 支持 Kali→Windows 真实攻击答辩。

成员贡献摘要：李哲（capture、108 条初版 mock、PR#1）；曾子恒（12 条基础规则、特征库管理、行为聚合、PR#3；主分支后续扩展至 28 条规则）；陈志恒（login_monitor.py、SYN/RST 与 flow_id 归集、双指针滑动窗口、25 项测试，提交 ad8e877/c78abe2）；姜新晨（四项异常检测、PR#2）；韩宇飞（aggregator、correlator、Web GUI、main、426 条 mock 扩充与 demo）。

本工作证明：在五人课程项目中，「Schema 先行 + Mock 驱动 + 分 Phase 合并」可将联调风险前移，并在有限周期内交付可测试、可演示、可扩展的网络入侵检测系统。

参考文献

- [1] Denning, D. E. (1987). An intrusion-detection model. *IEEE TSE*, 13(2), 222–232.
- [2] Roesch, M. (1999). Snort: Lightweight intrusion detection for networks. *Proceedings of LISA '99*, 229–238.

- [3] MITRE ATT&CK Framework. <https://attack.mitre.org/> (2026-07-28).
- [4] OWASP Foundation. OWASP Top 10 – 2021. <https://owasp.org/www-project-top-ten/> (2026-07-28).
- [5] NIST. SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS). (2007).
- [6] Scapy Documentation. <https://scapy.readthedocs.io/> (2026-07-28).
- [7] interface_spec.md, network_detection repo (2026-07-28).

A 开发里程碑

日期	里程碑	负责人
7/9–7/12	mock + capture 骨架	李哲
7/13	PR #1 合并	李哲
7/13	bruteforce 核心逻辑与 25 项测试	陈志恒
7/14–7/21	signature/anomaly 核心检测逻辑	曾子恒、姜新晨
7/22	PR #2、PR #3 审核合并	姜新晨、曾子恒
7/24	TCP 重组 v2 与跨检测器协同	李哲、韩宇飞
7/25–7/27	跨机器演示与全链路联调	全员

B 五成员交付物清单

成员	核心文件	交付说明
李哲	src/capture/*, mock	108 条初版 mock、22 项测试 (20 通过/2 跳过)、PR#1
曾子恒	signature_engine/*	12 条基础规则、规则库管理、行为聚合、PR#3
陈志恒	bruteforce_detect/*	SYN/RST 与 flow_id 归集、双指针滑窗、25 项测试
姜新晨	anomaly_detect/*	扫描/外联/横向/高频连接、45 项测试、PR#2
韩宇飞	gui_alert/*, main	汇总、协同关联、双形态 GUI、426 条 mock 扩充、demo

C Verification

ID	断言	验证方式	结果
V1	章节覆盖引言/威胁/实现/实验/结论	结构检查	通过
V2	分工与 README 一致	对照 README	通过
V3	字段与 interface_spec 一致	对照规范	通过
V4	main commit be0c01e 可获取	GitHub 主分支快照	通过
V5	426 条报文产生 30 条告警	python main.py 实测	通过
V6	共收集 159 项测试	157 passed, 2 skipped	通过